CODE SPORT

Sandya Mannarswamy

In this month's column, we feature a set of computer science interview questions.

Last month, we had covered a set of questions on machine learning and natural language processing. We continue our discussion of computer science questions in this month's column, focusing on operating systems.

1. We are all familiar with different sorting and searching algorithms. However, these algorithms typically are designed for data that fits in physical memory. Consider that you are given two matrices, A and B, of dimensions MXN and NXR. These dimensions are such that neither of the matrices will fit in the main memory. You are asked to write an algorithm for multiplying the two matrices and storing the result into a matrix C of dimensions MXR on the disk. Can you outline the approach you would take to minimise the number of disk accesses that your *matrix_multiply* function would perform?
2. Now, with regard to this same problem, you are told that matrices A and B are such that most of their elements are zero. You are again asked to compute the matrix multiplication of these two matrices. Can you come up with an algorithm which minimises the number of disk accesses?
3. In operating systems, what is meant by page thrashing? When does it occur? How can you avoid page thrashing?
4. Consider the following code snippet:

```
int main(int argc, char *argv[])
{
        void *my_mem = 0;
        int alloc_blocks = 0;

        while(1)
        {
                my_mem = (void *) malloc(1024 * 1024);
                if (!my_mem) break;
                memset(my_mem,1, (1024*1024));
                printf("allocated memory blocks is %d MB\n",++alloc_blocks);
        }
        exit(0);

}
```

Can you explain what would happen when you compile and run this program?

5. Consider this same code snippet and assume that you are asked to modify the code so as to remove the call to the memset function. Can you explain what would happen when you compile and run the code?
6. On your Linux machine, you see the following message on the */var/log/messages* file: "kernel: Out of memory: Kill process 2000 (a.out) score 511 or sacrifice child"
   - How can you figure out why Process 2000 was chosen to be terminated?
   - If you want to make the termination of Process 2000 unlikely, how would you do that?
   - In the scenario above, only the Process 'a.out' with the PID 2000 got terminated. Instead, you want to configure your system such that the OS gets restarted whenever an out-of-memory condition occurs. How would you achieve this?
7. Linux offers different kernel memory allocators like the standard SLAB allocator, the SLUB allocator and the SLOB (Simple List Of Blocks) allocator. You are asked to set up a Linux system which has been configured with 4GB of physical memory. Which memory allocator would you use and why?
8. Looking at Question 7 further, you are asked to configure the Linux kernel for an embedded system with a physical memory of only 256MB. What would be your choice of memory allocator? Can you explain the rationale behind your choice?
9. Consider that you are running your application on a Linux system that has 4GB of physical memory.